SAEP

Solana Agent Economy Protocol

FRONTEND BUILD DOCUMENT

Developer Portal | Agent Dashboard | Task Marketplace | Analytics Console

ENGINEERING SPECIFICATION | APRIL 2026 | INTERNAL

1. Frontend Architecture Overview

The SAEP frontend is the primary human-facing layer of the Solana Agent Economy Protocol. It serves four distinct user classes — agent developers, enterprise operators, arbitrators, and protocol governors — through a unified Next.js 15 application. Every screen is a live window into on-chain state; no data is mocked or cached beyond standard RPC query TTLs.

> *Design principle: The SAEP frontend must feel as fast as Web2. With Alpenglow delivering sub-150ms block finality and Firedancer sustaining 100,000+ TPS, the blockchain is no longer the bottleneck. The frontend must be engineered to match that speed — server components, streaming SSR, optimistic UI, and Yellowstone gRPC subscriptions for real-time state.*

1.1 Technology Stack — Full Specification

Layer	Technology	Version	Rationale
Framework	Next.js 15 (App Router)	15.x	Server Components + streaming SSR eliminate loading states; layouts co-locate with on-chain data fetching
UI Library	React 19	19.x	Concurrent rendering, useTransition for optimistic wallet interactions, Actions API for form handling
Styling	Tailwind CSS 4 + shadcn/ui	4.x / latest	Utility-first; design tokens map directly to SAEP brand; shadcn primitives for accessible components
State Management	Zustand + React Query (TanStack)	5.x / 5.x	Zustand for wallet/session state; React Query for on-chain data with 400ms stale TTL matching Alpenglow slot time
Wallet Adapter	@solana/wallet-adapter-react	0.15+	Phantom, Backpack, Solflare, Ledger; Wallet Standard 1.0 for future hardware wallet support
Web3 Client	@solana/web3.js v2 (legacy compat)	2.x	New functional API; tree-shakeable; 40% smaller bundle vs v1
Anchor Client	@coral-xyz/anchor	0.30+	Type-safe IDL-driven calls to all 6 SAEP programs; auto-generates TypeScript types from Rust IDL
Real-time	Yellowstone gRPC (Helius)	latest	Slot streaming + account subscriptions for live agent status, task updates, escrow state changes
Charts	Recharts + D3.js	2.x / 7.x	Recharts for standard dashboards; D3 for agent network graph visualization
Testing	Vitest + Playwright	latest	Unit: Vitest for component/hook logic; E2E: Playwright for full wallet interaction flows on

			devnet
Auth	SIWS (Sign-In With Solana)	latest	Wallet-native session auth; no passwords; operator sessions scoped to connected pubkey
Deployment	Vercel Edge Network	latest	Edge functions for RPC proxying; ISR for static registry pages; zero cold starts

1.2 Application Structure

The SAEP frontend is organized as a monorepo using Turborepo with the following application packages:

Package	Route Prefix	Description
apps/portal	/	Primary app: agent dashboard, task marketplace, operator console, governance
apps/docs	/docs	Developer documentation site: SDK reference, protocol guides, tutorials
apps/analytics	/analytics	Public protocol analytics: fee burns, agent activity, task volume, token metrics
packages/ui	—	Shared component library: SAEP design system, wallet components, data tables
packages/sdk-ui	—	React hooks wrapping SAEP TypeScript SDK for frontend consumption
packages/config	—	Shared ESLint, TypeScript, Tailwind configs

2. Pages & Component Specification

2.1 Developer Portal — Agent Management

/dashboard — Operator Home

The central hub for operators managing their agent fleets. Renders immediately via Server Components with fresh on-chain data. Real-time updates via Yellowstone account subscriptions.

- AgentFleetGrid: Card grid of all registered agents. Each card shows: DID, capability tags, reputation score (0-10,000), current status (Active/Paused/Suspended), treasury balance, 24h jobs completed. Clicking opens AgentDetailSheet.
- TreasuryOverviewPanel: Aggregated balance across all agent PDAs. Shows SOL, USDC, SAEP token. One-click 'Top Up' opens Treasury modal with Jupiter swap integration for auto-converting any SPL token.
- FleetActivityFeed: Live Yellowstone-subscribed event log. TaskCreated, TaskCompleted, PaymentReceived, DisputeRaised events stream in real time. Timestamp, agent, counterparty, amount shown per event.
- GovernanceAlerts: Pending proposals, parameter changes near voting deadline, slashing events. Dismissible. Links to /governance for action.

/agents/register — Agent Registration

Multi-step form wizard for registering a new agent on-chain. Calls AgentRegistry.register_agent on completion.

1. Step 1 — Identity: Agent name, description, DID prefix, operator wallet binding. DID preview auto-generates as user types.
2. Step 2 — Capabilities: Multi-select tag picker from CapabilityRegistry on-chain list. Tags rendered as colored chips. Custom capability request form for new tags (goes to governance).
3. Step 3 — Pricing: Per-task price in USDC or SAEP. Optional streaming rate (per-second for compute tasks). Spending limit configuration for the agent's own outbound payments.
4. Step 4 — Stake: SAEP stake amount slider. Shows real-time discovery priority rank and estimated yield. Minimum stake validation pulled from GovernanceCore parameters.
5. Step 5 — Review & Deploy: Full summary. Shows estimated on-chain cost (registration fee + rent). Simulates transaction via simulateTransaction before submit. Wallet sign + broadcast.

/agents/[did] — Agent Detail

Deep-dive view for a single agent. Fully server-rendered with live Yellowstone subscription for real-time updates.

- CapabilityManifestViewer: JSON viewer for the agent's IPFS/Arweave capability manifest. Shows input/output schema, SLA commitments, pricing tiers.

- ReputationRadarChart: D3 radar chart visualizing 6 reputation dimensions: Task Completion, Payment Reliability, Response Time, Capability Accuracy, Dispute Rate, Stake Score. Benchmarked against protocol median.
- JobHistoryTable: Paginated table of all completed jobs. Sortable by date, value, duration. Expandable rows show task hash, proof reference, settlement transaction.
- TreasuryTimeline: 30-day sparkline of agent balance. Income vs. outgoing payments. Hover reveals per-day breakdown.
- EditCapabilities / UpdatePricing: Owner-only controls. Inline form with optimistic UI — updates reflected immediately, reverted on tx failure.

2.2 Task Marketplace

/marketplace — Task Discovery

The primary surface for clients (human or agent) to discover and hire agents. Optimized for both human browsing and programmatic access via the Discovery API.

- CapabilityFilterPanel: Left sidebar with tag checkboxes (all capability tags from CapabilityRegistry). Price range slider. Reputation minimum slider. Response time SLA dropdown. 'Available Now' toggle.
- AgentResultsGrid: Main content area. Sorted by: Best Match (default), Reputation, Price, Speed. Each card: agent name, top 3 capability chips, reputation composite score, starting price, avg response time, 'Hire' CTA button.
- QuickHireModal: Triggered by 'Hire' button. Task description textarea with hash preview. Payment amount input with live fee calculation (0.1% protocol + 0.05% SolRep pool). Escrow funding breakdown. Deadline picker. One-click 'Create Task Contract' — signs and broadcasts atomically.
- A2A Discovery Panel: Toggle showing agents filtered by A2A-compatible capabilities. Designed for orchestrator agents browsing on behalf of pipelines. Same UX but surfaced differently for developer context.

/tasks/[id] — Task Detail

Full lifecycle view of a single task contract. Live state machine visualization.

- TaskStateTimeline: Visual stepper: Created → Funded → InExecution → ProofSubmitted → Verified → Released (or Disputed). Active step highlighted. Timestamps per transition.
- EscrowPanel: Current escrow balance. Client and agent addresses (abbreviated, click-to-copy). Payment breakdown: task value, protocol fee, SolRep fee, net to agent.
- ProofViewer: When proof submitted — shows Groth16 proof hash, verification status from ProofVerifier program, ZK circuit used, verification tx link on Solscan.
- DisputePanel: Active only during dispute window (24h post-completion). 'Raise Dispute' button for client. Shows arbitrator pool, staked collateral, estimated resolution time. Links to DisputeArbitration program state.

2.3 Treasury & Payments Console

/treasury — Operator Treasury

- MultiAssetBalanceTable: All agent PDAs in fleet. Columns: Agent, SOL balance, USDC balance, SAEP balance, wXRP balance (Phase 3+). Bulk 'Rebalance' action auto-routes Jupiter swaps to normalize balances.
- StreamingPaymentsMonitor: Real-time table of active payment streams. Per-stream: counterparty, rate (tokens/second), elapsed, total streamed, 'Cancel Stream' action.
- SpendingLimitsEditor: Per-agent spending limit controls. Daily limit, weekly limit, per-transaction max. Enforced at TreasuryStandard program level. Changes require wallet signature.
- ExportReports: CSV export of all treasury activity. Date range picker. Useful for accounting, enterprise tax reporting.

2.4 Analytics Dashboard

/analytics — Protocol Analytics (Public)

- FeesBurnedCounter: Animated counter showing cumulative SAEP burned. Live Yellowstone subscription updates per-block. 7-day and 30-day burn rate sparklines.
- AgentEconomyMap: D3 force-directed graph of agent-to-agent transaction relationships. Node size = task volume. Edge thickness = payment frequency. Zoom/pan enabled. Cluster by capability category.
- TaskVolumeChart: Recharts area chart. Daily task count, task value (USDC), protocol fee revenue. Grouped by capability category. 7d / 30d / 90d toggles.
- TopAgentsLeaderboard: Top 20 agents by: jobs completed, total earned, reputation score. Toggleable ranking. Agent names link to /agents/[did].
- NetworkHealthPanel: Solana stats pulled from Helius: current TPS, slot time, finality time (shows Alpenglow upgrade status), Firedancer stake percentage.

2.5 Governance Portal

/governance — Protocol Governance

- ActiveProposalsGrid: Card list of open proposals. Shows: title, proposer, current vote tally (Yes/No/Abstain), quorum progress bar, time remaining, parameter being changed. 'Vote' button for token holders.
- VotingModal: Shows proposal detail, economic impact analysis, voting weight (based on SAEP staked). One-click vote with wallet signature. Confirms optimistically, verifies on-chain.
- ProposalHistory: Past proposals with outcomes. Filterable by type: fee change, capability addition, slashing parameter, treasury grant.

- CreateProposalForm: Structured form for new proposals. Proposal type selector auto-populates the parameter being changed. Requires minimum stake to submit (pulled from GovernanceCore).

3. Wallet & Web3 Integration

3.1 Wallet Connection Architecture

SAEP uses the Solana Wallet Standard 1.0 for wallet integration, supporting all compliant wallets (Phantom, Backpack, Solflare, Ledger). The wallet adapter context wraps the entire application at the root layout level.

```
// Root layout wallet context
// apps/portal/app/layout.tsx
import { WalletContextProvider } from '@/components/wallet/WalletContextProvider';
export default function RootLayout({ children }) {
  return <WalletContextProvider>{children}</WalletContextProvider>;
}
```

The WalletContextProvider initializes wallet adapters, handles auto-connect from localStorage, and exposes useWallet() + useConnection() hooks throughout the app tree.

3.2 Sign-In With Solana (SIWS) Session Flow

6. User connects wallet — triggers standard wallet adapter connection
7. Frontend requests SIWS nonce from /api/auth/nonce (server-generated, 5-min TTL)
8. Constructs SIWSMessage with domain, address, nonce, expiration
9. User signs message via wallet adapter — no SOL spent, no on-chain tx
10. Frontend sends signed message to /api/auth/verify — server verifies signature
11. Server sets httpOnly session cookie scoped to wallet pubkey
12. All subsequent operator API calls authenticated via session cookie — no re-signing needed

3.3 Transaction Lifecycle — Optimistic UI Pattern

All on-chain writes follow the same optimistic UI lifecycle to maintain web2 responsiveness:

13. User submits form / clicks action — UI immediately reflects intended state (optimistic update via React Query mutation)
14. Transaction built client-side using Anchor IDL-generated instruction builders
15. simulateTransaction() called first — if simulation fails, optimistic update is rolled back with error toast
16. Transaction signed by wallet adapter
17. Sent via Helius staked connection for priority processing — bypasses standard mempool congestion
18. Yellowstone gRPC subscription fires on account update — React Query cache invalidated

19. UI confirms final on-chain state — optimistic state replaced with verified state

3.4 Jito Bundle Integration

For atomic multi-step operations (e.g., Task contract creation + escrow funding + agent notification in one bundle), SAEP frontend uses the Jito Block Engine SDK:

```
import { searcherClient } from 'jito-ts/dist/sdk/block-engine/searcher';
const bundle = new Bundle([ix_create_task, ix_fund_escrow, ix_notify_agent], 10);
await client.sendBundle(bundle);
```

Jito bundles guarantee atomicity — either all three instructions land in the same block or none do. This is critical for task contract creation where partial state (task created but not funded) would leave the escrow in an undefined state.

4. Real-Time Data Architecture

4.1 Yellowstone gRPC Subscriptions

SAEP's real-time UI is powered by Yellowstone gRPC from Helius, providing slot-by-slot account update streaming. This is critical for agent status, task lifecycle, and treasury balance accuracy.

Subscription	Account / Program	Update Trigger	UI Component Updated
Agent Status	AgentRegistry PDAs	status field change	FleetActivityFeed, AgentFleetGrid status chips
Task Lifecycle	TaskMarket PDAs	task state machine transition	TaskStateTimeline, task list real-time status
Escrow Balance	TreasuryStandard PDAs	balance change (any SPL token)	TreasuryPanel, AgentCard balance display
Fee Burns	FeeCollector Program	burn instruction executed	FeesBurnedCounter animated increment
Governance Votes	GovernanceProgram PDAs	vote cast, tally update	ActiveProposalsGrid vote counts
Dispute Events	DisputeArbitration PDAs	arbitrator assignment, resolution	DisputePanel, task detail notification
Network Health	Helius slot stream	every slot (~400ms)	NetworkHealthPanel TPS, finality time

4.2 React Query Cache Strategy

SAEP uses React Query (TanStack Query v5) for all on-chain data fetching with TTLs calibrated to Alpenglow's finality window:

Query Type	staleTime	gcTime	Rationale
Agent list / discovery	400ms	5 min	Matches Alpenglow slot time — data is always at most 1 block old
Single agent detail	400ms	5 min	Reputation scores and balances change per task
Task contract state	0ms	2 min	Always fresh — task state is critical path for payments
Protocol analytics	30s	10 min	Aggregate metrics — slight staleness acceptable
Governance proposals	5s	10 min	Voting tallies update infrequently

			between votes
SAEP token price	10s	1 min	Price-sensitive but not block-by-block critical
Capability tags list	5 min	1 hr	Rarely changes — governance-gated additions only

5. SAEP SDK-UI Hooks Library

The packages/sdk-ui package wraps the SAEP TypeScript SDK in React hooks, making on-chain interactions feel like standard React state. All hooks use React Query under the hood.

5.1 Agent Hooks

Hook	Parameters	Returns	Description
useAgent(did)	did: string	AgentAccount | undefined	Fetch single agent PDA. Real-time via Yellowstone if subscribed.
useAgentList(filter?)	CapabilityFilter	AgentAccount[]	Filtered list from registry. Paginated, sortable.
useRegisterAgent()	—	{ mutate, isLoading, tx }	Mutation hook for AgentRegistry.register_agent
useUpdateManifest()	—	{ mutate, isLoading, tx }	Owner-only manifest update. Optimistic state.
useAgentReputation(did)	did: string	ReputationScore	6-dimension reputation score, benchmarks, history

5.2 Task & Escrow Hooks

Hook	Parameters	Returns	Description
useTask(taskId)	taskId: string	TaskContract | undefined	Full task state. Yellowstone real-time if live task.
useCreateTask()	—	{ mutate, isLoading, tx }	Calls TaskMarket.create_task + NexusEscrow.fund via Jito bundle.
useSubmitResult()	—	{ mutate, isLoading, tx }	Agent-side: submits result hash + Groth16 proof.
useRaiseDispute()	—	{ mutate, isLoading, tx }	Client-side dispute initiation within dispute window.
useTreasuryBalance(did)	did: string	{ sol, usdc, saep, total }	Multi-asset balance from TreasuryStandard PDA. Live.
useStreamPayment()	—	{ start, stop, status }	Initiate/cancel streaming

			payment channel.

6. Performance & Optimization Strategy

6.1 Solana-Aware Rendering Optimization

Standard web performance best practices apply, but SAEP adds Solana-specific optimizations that account for block-time data freshness and wallet interaction latency:

- Server Components for registry data: AgentList, CapabilityTags, and GovernanceProposals render as React Server Components. Initial HTML includes fully-hydrated data from Helius RPC — no loading spinners on first paint.
- Alpenglow-synchronized refetch: React Query refetch intervals are set to 400ms for critical data, matching Alpenglow's block slot time. This ensures UI is always at most one block stale.
- Prefetch on hover: Hovering over an agent card prefetches the full agent detail page via Next.js prefetch(). By the time the user clicks, the page is ready.
- RPC request deduplication: Multiple components requesting the same account within 50ms are automatically batched by React Query's deduplification. Reduces Helius RPC credit consumption by ~60%.
- Address Lookup Tables (ALTs): Complex multi-instruction transactions (task creation bundles) use ALTs to stay within Solana's 1232-byte transaction size limit. ALTs are pre-built and cached.
- Compute Unit budgeting: All transactions include ComputeBudgetProgram.setComputeUnitLimit() and setComputeUnitPrice() calls. CU limit is set to simulation result + 10% buffer. Priority fee is dynamically set based on Helius priority fee API.

6.2 Bundle Size Targets

Bundle	Target Size (gzipped)	Strategy
Initial JS (above the fold)	< 80 KB	Server Components, code splitting; wallet adapter lazy-loaded post-connect
@solana/web3.js v2	< 40 KB	Tree-shakeable functional API; only import used functions
Anchor client + IDLs	< 30 KB	IDLs loaded dynamically per program; not bundled in initial chunk
Charts (Recharts + D3)	< 60 KB	Dynamic import on analytics route; not included in main bundle
Total first-load JS	< 200 KB	Verified via Vercel bundle analyzer in CI pipeline

7. Frontend Security Model

7.1 Wallet & Transaction Security

- Transaction simulation before signing: All transactions are simulated via simulateTransaction() before presenting to the user for signing. If simulation returns an error, the UI surfaces a human-readable error before the wallet dialog opens — preventing users from signing transactions that will fail.
- No private key handling: SAEP frontend never requests, stores, or transmits private keys. All signing is delegated to the wallet adapter. The app is purely a transaction builder.
- RPC proxying via Vercel Edge: All Helius RPC calls route through /api/rpc edge function — API keys never exposed to client. Rate-limiting enforced per wallet pubkey.
- SIWS session scoping: Sessions are cryptographically bound to the operator's wallet pubkey. Cross-wallet data is never accessible within a session.
- Content Security Policy: Strict CSP headers disallow inline scripts, restrict fetch origins to known RPC endpoints, and prevent clickjacking via frame-ancestors 'none'.

7.2 Input Validation & Sanitization

- All user-provided strings (agent names, task descriptions) are sanitized with DOMPurify before display to prevent XSS from on-chain data
- Numeric inputs (prices, stake amounts, limits) validated client-side against BN.js bounds (u64 max) before transaction construction
- DID format validated against regex before registry lookup — prevents malformed PDA derivation
- Task description hash computed client-side and verified matches on-chain hash before displaying task details

8. Testing Strategy

8.1 Unit & Integration Tests (Vitest)

Test Suite	Coverage Target	Key Test Cases
SDK-UI hooks	95%	useAgent returns correct data; mutation hooks update optimistic state; Yellowstone subscription updates cache
Transaction builders	100%	Every instruction builder: correct accounts, correct data, CU budget set, ALT used where required
Wallet adapter mocks	90%	SIWS flow end-to-end; sign/reject flows; session expiry handling
Form validation	100%	All registration form steps; edge cases: max string lengths, u64 overflow, invalid DIDs

8.2 End-to-End Tests (Playwright)

Playwright tests run against a local validator with SAEP programs deployed (anchor localnet). Devnet smoke tests run on every PR merge.

- Full agent registration flow: Connect wallet → fill 5-step form → sign tx → verify agent appears in registry
- Hire agent and complete task: Browse marketplace → hire agent → simulate task completion → verify escrow release
- Dispute flow: Create task → raise dispute → simulate arbitration → verify collateral distribution
- Governance vote: Connect staker wallet → vote on proposal → verify tally updates in real time
- Treasury top-up: Open treasury modal → input amount → simulate Jupiter swap → verify balance update